Pandas & LLM Safety Guide (ED Coordinator Project)

Goal: prevent 'looks-fine-but-wrong' failures when using Pandas and large language models (LLM
a safety-critical coordination pipeline. This is a concise, practical checklist.

Use this when reviewing notebooks/PRs and when sanity-checking synthetic runs. Keep human o
on clinical realism.

Stage-by-Stage Guardrails (Implement in Order)

Stage 1 — Don’t trip over obvious wires (week 1–2):

- Pandas: use .convert_dtypes(); timestamps as UTC; forbid chained indexing; assert encounter_i uniqueness.

- LLMs: require structured JSON with fixed keys (action, params, requires_permit, justification). Reject anything else.

Stage 2 — Catch silent killers (after first synthetic end-to-end run):

- Pandas: merges must include validate='1:1' or '1:m'; explicit .copy() before modifying slices.

- LLMs: every proposal must include requires_permit and a short justification (features + rules_fired).

Stage 3 — Make it hard to regress (once flows are stable):

- Track KPIs: door_to_CT, door_to_first_lab, blocked_unsafe_actions, duplicate_proposals.

- Keep KPI history; investigate regressions before feature changes.

Stage 4 — Lock in trust (when sharing/testing more broadly):

- Sign events; anchor hashes off-chain/on-chain; keep propose→permit→execute with post-hoc re for time-critical exceptions.

Checklist order prevents most high-impact mistakes with minimal overhead.

Pandas: Common Failure Modes → Fixes

- Chained indexing & ambiguous selection: df[mask]['col']=... silently writes to a copy.

  Fix: one-shot .loc; if slicing then mutating, use .copy() first.

```
m = df.a > 0
df.loc[m, 'b'] = value          # good
sub = df.loc[m, ['b']].copy()   # explicit copy before mutate
```

- Naive to_datetime and tz chaos: missing utc=True mixes timezones and breaks ordering.

  Fix: normalize at ingest; assert monotonic per encounter.

```
df['ts'] = pd.to_datetime(df['ts'], utc=True, errors='raise')
assert df.sort_values(['encounter_id','ts'])['ts'].groupby(df['encounter_id']).apply(lambda s: s.
```

- Wrong merges: unintended many-to-many creates duplicated rows and wrong counts.

  Fix: always set validate=... and pre-dedupe keys.

```
joined = a.merge(b, on='encounter_id', how='left', validate='m:1')
```

- dtype drift & missing data: numbers become strings; object columns sneak in.

  Fix: .convert_dtypes(); define a schema; assert after load.

- inplace=True and deprecated indexers: hide copy bugs (.ix) and make diffs harder to reason ab

  Fix: avoid inplace=True; reassign; never use .ix.

LLMs: Failure Modes → Fixes (Proposer, Not Oracle)

- Fluent wrongness (hallucinations): outputs look plausible but are incorrect or incomplete.

  Fix: enforce a strict JSON schema, allow-list actions, and validate before any effect.

```
{
  "action": "order_ct",
  "params": {"protocol": "head_trauma", "priority": "STAT"},
  "requires_permit": true,
  "justification": {
    "features": {"gcs": 12, "on_ac": true},
    "rules_fired": ["trauma.prealert","ct.head.if_anticoagulated"],
    "confidence": 0.84
  }
}
```

- Non-determinism & prompt fragility: small phrasing changes shift behavior.

  Fix: low temperature for safety-critical proposals; keep a small explicit state JSON; avoid long narrative prompts.

- Invented labels & enums: LLMs create novel category values.

  Fix: hard-code category vocab; map UNKNOWN/OTHER explicitly; reject unseen labels.

- Overreach: executing high-risk actions without human review.

  Fix: require permits for admissions, ICU allocations, blood products, thrombolytics, OR activation; time-critical exceptions only with post-hoc review.

ED Coordination: Invariants & Contracts

- Identity: no orders without a bound encounter_id (except explicit time-critical override).

- Time: compute door_to_CT and door_to_first_lab; enforce policy thresholds or escalate.

- Capacity: ICU cannot go below 0; when ICU is full, only ED hold or transfer are valid proposals.

- No duplicates: idempotency key = (encounter_id, action, params_hash); cooldowns suppress sp

- Permit boundaries: high-risk actions are propose→permit→execute with justification.

```
idempotency_key = sha256(json.dumps({
  'enc': encounter_id,
  'action': 'order_ct',
  'params': {'protocol':'head_trauma','priority':'STAT'}
}, sort_keys=True)).hexdigest()
```

Safe Patterns (Drop-in)

Pandas load & basic checks:

```
df = pd.read_csv(path).convert_dtypes()
df['ts'] = pd.to_datetime(df['ts'], utc=True, errors='raise')
assert df['encounter_id'].is_unique
```

Merge with guarantees:

```
orders = orders.merge(patients[['encounter_id','triage_class']],
                      on='encounter_id', how='left', validate='m:1')
```

LLM action schema (validator expects these keys):

```
{
  "action": "order_ecg | order_ct | request_labs | bed.request | ed_hold",
  "params": { "...": "..." },
  "requires_permit": true,
  "justification": { "features": {...}, "rules_fired": [...], "confidence": 0.xx }
}
```

Repeat policies (examples): CT once per encounter; ECG/Labs every 2–3h; `new_indication=true` overrides CT lockout.

AI-Generated Code: Tells & Review Heuristics

- Stylistic tells: verbose docstrings, friendly generic error handling, consistent naming, safe defaults.

- Structural gaps: lack of edge-case handling, missing merge validate, naive datetime handling, broad try/except.

- Testing tells: trivial ‘happy path’ tests; no negative tests; no KPI checks.

- Data leakage risk: fitting scalers/encoders on full data; using test info in train-time preprocessing.

- Reproducibility gaps: no fixed seeds; not logging model/config/version hashes.

Review moves:

- Ask for invariants (what must always be true) and check them immediately after risky ops.

- Require validator passes and justifications for every high-risk action proposal.

- Reject PRs without KPI deltas and a one-page behavior diff on 2 synthetic cases.

Debug Playbook (When Something Looks Off)

1) Is the data frame what you think? → print shapes, dtypes, unique keys; assert schema.

2) Did a write actually mutate? → check counts before/after; avoid chained indexing; assert invariants.

3) Was a merge legal? → validate='1:1'/'1:m'; check row counts vs expectation.

4) Is the LLM proposal valid? → run through validator; check allow-list and required fields.

5) Are you seeing spammy actions? → add cooldown + idempotency; switch to edge-triggered ev

6) Did a KPI regress? → stop and bisect changes; revert if necessary.